2.1 Morphology fundamentals; Morphological Diversity of Indian languages

Morphology is the structure of a word as a composition of morphemes. A morpheme is the smallest unit of language that carries meaning. For example: unhappiness = [un][happy][ness]. Each of these is a morpheme and each contributes to the final meaning, which is "the state of not being happy." This is exactly what morphology studies: how words are built from these meaningful building blocks.

The three core functions of morphology are: inflection, derivation, and composition (compounding).

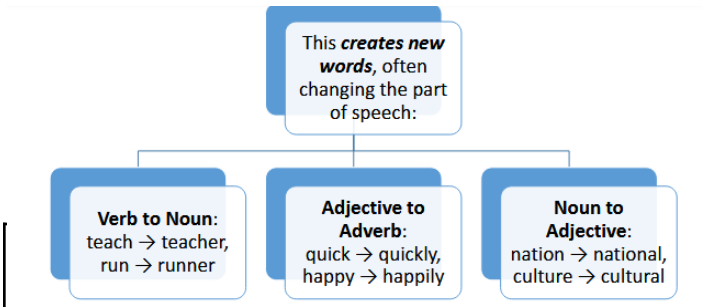
Inflection

It is the morphological process that modifies a word to express grammatical features such as tense, number, person, case, or degree, without changing the word's core meaning or part of speech. It allows words to fit correctly into sentences while preserving their identity. These changes do not create new words but simply adjust existing ones for grammatical accuracy. For example:

- **Number**: house → houses, child → children
- **Tense**: walk → walked → walking

English only has 8 inflectional morphemes.

-s (plural) → cats	-'s (possessive) → cat's	-ed (past tense) → talked	-en (past participle) → taken
-ing (progressive) → taking	-s (rd person singular) → takes	-er (comparative) → taller	-est (superlative) → tallest



Derivation

Derivation is the process of creating new words by adding prefixes or suffixes, which often change the meaning of the base word and sometimes its grammatical category. Unlike inflection, derivation expands the vocabulary and allows words to take on new functions. Through derivation, language develops new words to express different ideas.

verb → adjective: talk → talkative	**verb → noun**: teach → teacher	**noun → noun**: child → childhood
noun → adjective: danger → dangerous	**adjective → adjective**: kind → unkind	**adjective → adverb**: quick → quickly

Composition

Composition is a morphological process where two or more independent words are combined to form a single new word with a distinct meaning. This method does not rely on affixes but instead uses existing words as building blocks to create new lexical items. These compounds take on meanings that may extend beyond the sum of their parts, enriching the vocabulary of a language in creative and practical ways. Examples:

- **English:** black+board, key+board, soft+ware
- **German**: computerwisschenschaft (computer + applied science)

Types of morphemes

1. **Free morphemes** are the smallest units of meaning that can stand alone as independent words in a language. They do not need to attach to other morphemes to make sense. For example, words like book, cat, run, and happy are all free morphemes because they can function as complete words on their own. These morphemes provide the core vocabulary and structure of a language.
2. **Bound morphemes**, on the other hand, cannot stand alone as words and must be attached to a free morpheme to create meaning. They are typically prefixes, suffixes, or inflectional endings. For instance, -s in cats (plural), -ed in played (past tense), un- in unhappy (negation), and -ness in kindness are all bound morphemes. They add

grammatical information or change word class/meaning. Bound morphemes are essential for expressing nuances in grammar and for creating new words through morphological processes.

Morphological analysis is the study of the internal structure of words and how they are formed by combining smaller meaningful units called morphemes. It focuses on identifying roots, prefixes, and suffixes, as well as understanding how these elements interact to create new words or modify existing ones. Through morphological analysis, linguists can explain how words convey grammatical information (such as tense, number, or case) and how new vocabulary is derived or compounded. For example, analyzing the word unhappiness reveals three morphemes: the prefix un- (negation), the root happy (base meaning), and the suffix -ness (state/quality), which together form a noun meaning "the state of not being happy."

Morphological analysis: the computational challenge

1. Phonological alterations

- Words often undergo sound changes when morphemes are combined, making analysis complex.
- **Example**: *happy* + *ness* → *happiness* (the *y* changes to *i*).
- **Example**: *electric* + *ity* → *electricity* (stress pattern shifts).

2. Morphotactic constraints

- Rules that govern how morphemes can be combined within a language.
- Example: In English, plural *-s* attaches to nouns (*cat* → *cats*), not verbs (*run* → *runs* is agreement, not pluralization).
- Some affixes only combine with specific word classes (e.g., *-ness* attaches to adjectives → *kindness*, but not to verbs like *runness*).

3. Ambiguity

- A single surface form can correspond to multiple morphological analyses.
- Example: *unlockable* can mean "not lockable" (*un-* + *lockable*) or "able to be unlocked" (*unlock* + *-able*).
- **Example**: *flies* could be a plural noun (*the flies*) or a verb form (*she flies*).

Morphological analysis techniques

1. Rule-based approaches

- Rely on explicitly defined linguistic rules for morpheme segmentation and analysis.
- Use lexicons, affix lists, and morphotactic constraints to determine valid word forms.
- Effective for well-documented, less complex languages.
- **Example**: Finite State Transducers (FSTs) that map surface forms (*cats*) to underlying structure (*cat* + *PLURAL*).
- **Limitation**: Hard to scale for highly inflected or irregular languages; requires extensive manual work.

2. Statistical approaches

- Use probabilities and frequency patterns from large corpora to infer morphological structures.
- Segment words based on co-occurrence and likelihood of morpheme boundaries.
- **Example**: Unsupervised algorithms (like Morfessor) learn affix and root patterns from text.
- Better at handling ambiguity than rule-based systems.
- **Limitation**: Struggle with rare or unseen words; performance depends on size and quality of training data.

3. Neural network approaches

- Use deep learning models (e.g., RNNs, CNNs, Transformers) to learn morphological patterns from raw text.
- Can jointly model phonological changes, morphotactics, and disambiguation.
- **Example**: Sequence-to-sequence models that map words to their lemma + features (*running* → *run* + *PROG*).
- Highly effective for complex, morphologically rich languages.
- **Limitation**: Require large annotated datasets and high computational resources.

Stemming vs lemmatization

Aspect	**Stemming**	**Lemmatization**
Method	Rule-based chopping	Vocabulary + morphological analysis

Output	Stem (may not be a real word)	Lemma (dictionary word)
Speed	Fast	Slower (requires POS tagging)
Accuracy	Lower	Higher
Context awareness	No	Yes (considers POS)

Word formation processes

1. Affixation

- Adding prefixes, suffixes, infixes, or circumfixes to a base/root word.
- The most common word-formation process in English. Examples:
    - Prefix: un- + happy → unhappy
    - Suffix: teach + -er → teacher
    - Infix (rare in English, but seen in slang): fan-freaking-tastic

2. Compounding

- Combining two or more independent words to form a new word.
- The new meaning may extend beyond the sum of the parts.
- **Examples**: toothbrush, blackboard, sunflower, laptop

3. Conversion (zero derivation)

- Changing the grammatical category of a word without altering its form.
- No affixes are added; the shift is functional. Examples:
    - **Noun → Verb**: Google (noun) → to Google (verb)
    - **Verb → Noun**: to run (verb) → a run (noun)
    - **Adjective → Noun**: poor (adj.) → the poor (noun)

4. Blending

- Merging parts of two words (often the beginning of one and the end of another).
- Results in a new word that combines both meanings.
- **Examples**: smoke + fog → smog, breakfast + lunch → brunch, motor + hotel → motel

5. Clipping

- Shortening a longer word without changing its meaning.
- Common in casual or colloquial usage.
- **Examples**: advertisement → ad, examination → exam, influenza → flu, telephone → phone

6. Acronymy

- Forming words from the initial letters of a phrase.
- Acronyms are pronounced as words; initialisms are pronounced letter by letter. Examples:
    - **Acronym**: NATO (North Atlantic Treaty Organization), AIDS (Acquired Immune Deficiency Syndrome)
    - **Initialism**: FBI (Federal Bureau of Investigation), BBC (British Broadcasting Corporation)

7. Back-formation

- Creating a new word by removing an affix (real or imagined) from an existing word.
- Often results in a simpler form (usually a verb).
- **Examples**: editor → edit, television → televise, donation → donate, burglar → burgle

<u>Morphological diversity of Indian languages</u>

India presents one of the world's most complex morphological landscapes with:

- **4 major language families**: Indo-Aryan, Dravidian, Austro-Asiatic, Sino-Tibetan
- **22 official languages** plus hundreds of regional varieties
- **Diverse morphological types**: From relatively isolating to highly agglutinative

Key characteristics of Indian languages:

1. Rich morphological systems (compared to English)
2. Free word order
3. Complex verb systems with rich TAM (Tense-Aspect-Modality) marking
4. Agglutinative tendencies (especially Dravidian languages)

Challenges in processing Indian languages

- **Rich morphology**: Single words can express what English needs entire phrases for.
- **Free word order**: Grammatical relations encoded in morphology, not position.
- **Multiple scripts**.
- **Code-mixing**: Multiple languages in single texts.

Real-world applications

1. **Machine translation:** Morphological analysis plays a crucial role in machine translation by breaking words into their roots and affixes, which helps systems generate accurate equivalents in the target language. This is especially important for morphologically rich languages, where a single root may have hundreds of forms.
2. **Information retrieval:** In search engines and databases, morphological analysis helps improve results by reducing words to their base or root form. This ensures that different variations of a word are recognized as related. For instance, a query for running will also retrieve documents containing run or ran.
3. **Sentiment analysis:** Morphological analyzers assist sentiment analysis by identifying the core meaning of words and normalizing different inflected or derived forms. For example, happy, happier, and happiness all convey related sentiments and can be grouped together once reduced to the root happy. This improves the accuracy of models in detecting sentiment patterns, even when words appear in varied forms. It is especially helpful when dealing with social media texts, where users often use creative word forms.
4. **Text summarization:** In automatic text summarization, morphological analysis helps reduce redundancy and extract the most relevant content. By mapping inflected words to their base forms, systems can better identify key themes and avoid repetition of semantically similar terms. For example, recognizing that develop, developed, and development are related allows the summarizer to focus on the concept of development rather than treating each as separate. This enhances coherence and ensures summaries capture the essence of the original text.

2.2 Morphology paradigms, finite state machine based morphology, automatic morphology learning

What are morphological paradigms?

- A morphological paradigm is a complete set of related word forms associated with a given lexeme. Simply put, it's a systematic arrangement of all the different forms a word can take.
- Let’s understand with an example. Consider the English verb "walk": walk, walks, walked, walking.
- These four forms constitute the inflectional paradigm of the verb "walk". Each form serves a different grammatical function - present tense, third person singular, past tense, and present participle respectively.

Types of paradigms

1. Inflectional paradigms

- Represent the set of all possible inflected forms of a word.
- Show grammatical variations such as tense, number, case, gender, person, or degree.
- Do not create new words; they only modify the word form to fit grammatical context.
- Example (noun cat): cat, cats.
- Example (adjective big): big, bigger, biggest.
- Important in computational linguistics for tasks like lemmatization and POS tagging.

2. Derivational paradigms

- Represent the set of words derived from a single base/root using affixes.
- Often involve changes in meaning and sometimes in part of speech.
- Do create new lexical items, expanding vocabulary.
- Example (root nation): national, nationalize, nationality, nationalism.
- Example (verb teach): teacher, teaching, teachable.
- Important in understanding word formation, vocabulary growth, and semantic networks.

Computational significance

From a computational perspective, paradigms are crucial because they:

- **Reduce memory requirements** - instead of storing every word form, we store rules
- **Enable prediction** - if we know one form, we can generate others
- **Handle unknown words** - we can apply paradigmatic patterns to new vocabulary
- **Real-world application**: Google Search uses paradigmatic knowledge. When you search for "running shoes," it knows to also consider results about "run," "runs," and "ran" because these belong to the same paradigm.

Finite state machine based morphology

Finite State Machines (FSMs) provide an elegant mathematical framework for processing morphology. A Finite State Automaton (FSA) consists of:

- A finite set of states Q = {q_0, q_1, ..., $q_{\square}$}
- A finite alphabet Σ of input symbols
- A start state q_0
- A set of final states F
- A transition function δ

Two level morphology

The breakthrough came with Kimmo Koskenniemi's Two-Level Morphology in the 1980s. This system uses two levels:

- **Lexical level**: The underlying morphological representation
- **Surface level**: The actual word as it appears in text
- **Example**: Lexical: cat+N+Pl (cat=root morpheme, N=noun, Pl=plural, separated by +). The "+" symbols are morpheme boundaries that don't appear in the surface form.

Finite state transducers

FSTs extend FSAs by having both input and output tapes. They can transform one string into another, making them perfect for morphological analysis. FST Components:

- Input alphabet Σ (surface forms)
- Output alphabet Δ (lexical forms)
- Transition function mapping input:output pairs
- Output function generating morphological tags

Practical example - English plural formation

Lets understand how an FST would handle "foxes" → "fox+N+Pl“

State 1: f:f → State 1 : Reads *f* from the surface (*foxes*), outputs *f* in lexical form.

State 1: o:o → State 1 : Reads *o*, outputs *o*.

State 1: x:x → State 1 : Reads *x*, outputs *x*.

State 1: e:ε → State 2 (delete 'e') : Reads e from the surface, but outputs nothing (ε) in lexical form. This handles the extra e before -s in plurals like foxes.

State 2: s:+N+Pl → State 3 (final) : Reads s from surface, outputs the lexical tags: +N+Pl (noun + plural). The machine ends in the final state.

Here, ε represents an empty symbol, and the FST correctly identifies that "foxes" should be analyzed as "fox" + plural marker.

Morphotactics

Morphotactics defines the permissible combinations of morphemes. For example, in English:

- Prefix + Root + Suffix is valid: "un-break-able"
- *Root + Prefix + Suffix is invalid: "*break-un-able"

FSTs can encode these constraints elegantly through state transitions.

Spelling rules

Real morphology involves spelling changes. Consider:

- try + ing → trying (y→i change)

- take + ing → taking (e deletion)
- run + ing → running (consonant doubling)

FSTs handle these through orthographic rules that modify the surface representation.

Advantages of FST-based morphology

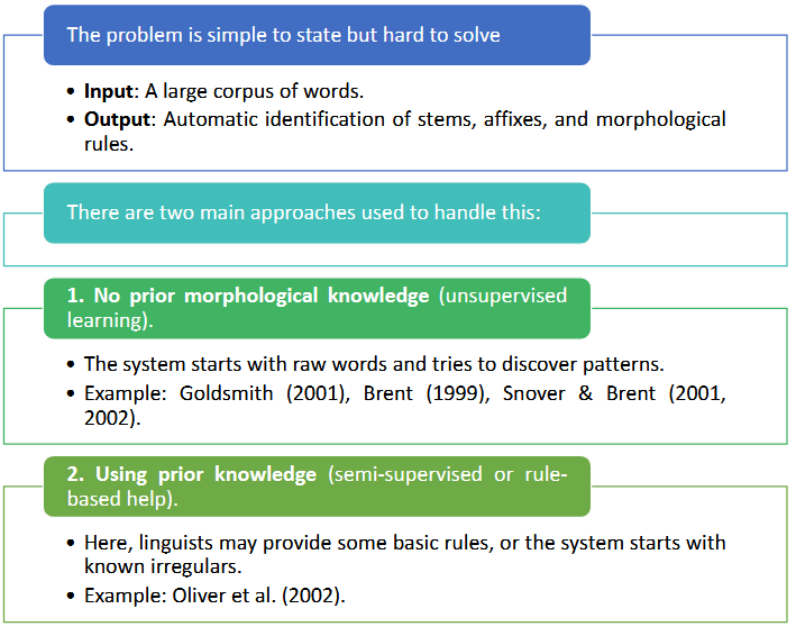
1. **Bidirectional**: Same FST can analyze (cats → cat+N+Pl) and generate (cat+N+Pl →cats)
2. **Efficient**: Linear time complexity for processing
3. **Compositional**: Multiple FSTs can be combined for complex morphological phenomena
4. **Language-independent framework**: The same mathematical model works across languages

Commercial applications

1. **Spell checkers**: Spell checkers use FSTs and dictionaries to detect and correct misspelled words in text. They suggest the most likely correct form based on edit distance or phonetic similarity.
2. **Machine translation systems**: Machine translation systems use morphology and FSTs to map words across languages while preserving meaning. They break down words into morphemes, translate them, and then generate correct surface forms in the target language.
3. **Information retrieval**: Information retrieval systems rely on stemming and morphological analysis to match different word forms (e.g., *run*, *running*, *ran*). This improves search accuracy by linking related terms and expanding queries.
4. **Grammar checkers**: Grammar checkers analyze syntax and morphology to detect grammatical errors in writing. They flag issues like subject-verb disagreement, tense errors, or incorrect word forms.

<u>Automatic Morphology Learning (AML)</u>

- We have already seen how words can be broken down into smaller meaningful units called morphemes. For example: 'unhappiness' → 'un-' (prefix) + 'happy' (stem) + '-ness' (suffix)
- The challenge in NLP is: How can a machine automatically learn these structures without being explicitly told?
- This is where Automatic Morphology Learning (AML) comes in. It tries to identify stems, affixes, and morphological rules directly from large text corpora.



How does AML work

1. Identification of morpheme boundaries

- Classic work by Zellig Harris used conditional entropy of prefixes and suffixes.
- **Idea**: If a certain prefix/suffix makes word endings very predictable, it's likely a true morpheme.
- **Example**:
    - In English, suffix '-ed' has high predictability: 'walked', 'played', 'jumped'.
    - In contrast, the sequence '-lp' rarely occurs, so it's not a valid suffix.

2. Statistical methods with bigrams and trigrams

- Systems look at frequent co-occurring patterns.
- If 'ing' often follows stems, the system marks it as a possible suffix.

3. Global (top-down) approaches

Goldsmith, Brent, and de Marcken proposed global statistical approaches: Instead of word-by-word guessing, they look for the best explanation of all words in the corpus together.

Semi-automatic morphological analysis

- Oliver (2004) proposed a semi-automatic method:
  - Starts with a set of manually written morphological rules.
  - Uses a format TL:TF:Desc → Lemma ending, Form ending, POS.
  - Lists irregular and closed-class words (like 'is', 'the', 'and').
- For example:
  - Rule: Verb stem ending in 'e' + 'ing' → drop 'e' + 'ing'.
  - Lemma: 'make' → Form: 'making'.
- This approach was tested on large corpora: Serbo-Croatian (9 million words), Russian (16 million words).

So, semi-automatic methods can scale better because they start with some linguistic knowledge instead of reinventing everything.

2.3 Shallow parsing, named entities, maximum entropy models, random fields

- Shallow parsing, also known as light parsing or chunking, is a natural language processing (NLP) technique focused on identifying and extracting key informational units or keywords from sentences without performing a full grammatical analysis.
- Unlike deep parsing, which creates a complete parse tree to capture the entire sentence structure, shallow parsing concentrates on recognizing broader linguistic components such as noun phrases, verb phrases, and prepositional phrases.
- The main objective of shallow parsing is to balance computational efficiency with linguistic accuracy. Rather than exploring the complex syntactic connections within a sentence, it aims to extract the most significant elements of the sentence structure for use in further NLP tasks.

Purpose of shallow parsing

1. **Information extraction**: Shallow parsing helps in extracting key elements from the text, such as named entities, noun phrases, and verb phrases, which are crucial for tasks like entity recognition, relationship extraction, and summarization.
2. **Text understanding**: By identifying important phrase fragments, shallow parsing aids in improving the analysis and understanding of text, enhancing its syntactic and semantic interpretation.
3. **Computational efficiency**: Compared to deep parsing, shallow parsing techniques are generally more computationally efficient, making them ideal for processing large amounts of text in real-time or near-real-time applications.
4. **Feature engineering**: For machine learning models involved in tasks such as text classification, sentiment analysis, and information retrieval, shallow parsing provides useful features by capturing higher-level textual structures.

Example:

- Sentence: "The cat sat on the mat."
- Shallow Parsing Output:
  - Noun Phrase (NP): "The cat"
  - Verb Phrase (VP): "sat"
  - Prepositional Phrase (PP): "on the mat"

Linguistic units of shallow parsing

1. **Words**: Each word in the sentence is evaluated for its function within a phrase and its grammatical category (e.g., part of speech).
2. **Phrases**: Phrases are groups of words that function as a unit within the sentence. Examples include noun phrases (NP), verb phrases (VP), and prepositional phrases (PP).

3. **Dependencies**: Shallow parsing can also identify dependencies between words or phrases, such as subject-verb or verb-object relationships.
4. **Named entities**: In some cases, shallow parsing algorithms can identify named entities, such as the names of people, organizations, or locations.

Common techniques

- **Part-of-Speech (POS) tagging**: This technique involves classifying the words in a sentence into grammatical categories, such as nouns, verbs, adjectives, etc. These categories, or tags, help determine the syntactic roles of words in a sentence.
- **Chunking**: Chunking is the process of grouping consecutive words in a sentence into meaningful chunks or phrases, such as noun phrases (NP) or verb phrases (VP). This technique often relies on POS tagging to define the boundaries of these chunks.
- **Named Entity Recognition (NER)**: NER identifies and categorizes named entities in text, such as names of places, organizations, and individuals. It extracts significant pieces of information from text, making it a form of shallow parsing.
- **Regular expressions**: Regular expressions are patterns used to recognize specific linguistic structures in text. These patterns can be applied to extract words or chunks based on predefined criteria.

Named entity recognition

Named Entity Recognition (NER) in NLP focuses on identifying and categorizing important information known as entities in text. These entities can be names of people, places, organizations, dates, etc. It helps in transforming unstructured text into structured information which helps in tasks like text summarization, knowledge graph creation and question answering.

NER Methods

1. Rule-based methods

- Rule-based methods are grounded in manually crafted rules. They identify and classify named entities based on linguistic patterns, regular expressions, or dictionaries.
- While they shine in specific domains where entities are well-defined, such as extracting standard medical terms from clinical notes, their scalability is limited. They might struggle with large or diverse datasets due to the rigidity of predefined rules.

2. Statistical methods

- Transitioning from manual rules, statistical methods employ models like Hidden Markov Models (HMM) or Conditional Random Fields (CRF).
- They predict named entities based on likelihoods derived from training data. These methods are apt for tasks with ample labeled datasets at their disposal. Their strength lies in generalizing across diverse texts, but they're only as good as the training data they're fed.

3. Machine Learning Methods

- Machine learning methods take it a step further by using algorithms such as decision trees or support vector machines. They learn from labeled data to predict named entities. Their widespread adoption in modern NER systems is attributed to their prowess in handling vast datasets and intricate patterns. However, they're hungry for substantial labeled data and can be computationally demanding.

NER use cases

1. **News aggregation**. NER is instrumental in categorizing news articles by the primary entities mentioned. This categorization aids readers in swiftly locating stories about specific people, places, or organizations, streamlining the news consumption process.
2. **Customer support**. Analyzing customer queries becomes more efficient with NER. Companies can swiftly pinpoint common issues related to specific products or services, ensuring that customer concerns are addressed promptly and effectively.
3. **Research**. For academics and researchers, NER is a boon. It allows them to scan vast volumes of text, identifying mentions of specific entities relevant to their studies. This automated extraction speeds up the research process and ensures comprehensive data analysis.
4. **Legal document analysis**. In the legal sector, sifting through lengthy documents to find relevant entities like names, dates, or locations can be tedious. NER automates this, making legal research and analysis more efficient.

Working of NER

1. **Analyzing the text**: It processes the entire text to locate words or phrases that could represent entities.
2. **Finding sentence boundaries**: It identifies starting and ending of sentences using punctuation and capitalization which helps in maintaining meaning and context of entities.
3. **Tokenizing and Part-of-Speech tagging**: Text is broken into tokens (words) and each token is tagged with its grammatical role which provides important clues for identifying entities.
4. **Entity detection and classification**: Tokens or groups of tokens that match patterns of known entities are recognized and classified into predefined categories like Person, Organization, Location etc.
5. **Model training and refinement**: Machine learning models are trained using labeled datasets and they improve over time by learning patterns and relationships between words.
6. **Adapting to new contexts**: A well-trained model can generalize to different languages, styles and unseen types of entities by learning from context.

Maximum entropy models

- Sentence: 'I am going to the bank.'
- Ambiguity: Bank = financial institution OR river side
- Features: Previous word = 'the', current word = 'bank'.
- MaxEnt uses all available clues without bias.

Philosophy of MaxEntropy

1. Among all models that fit data, choose the one with maximum entropy.
2. Entropy = uncertainty; maximizing it prevents adding bias.
3. Analogy: If you don't know about someone, don't assume extra things.

Applications in NLP

1. **Part-of-Speech tagging**: Predicting tags using features like suffix, capitalization, surrounding words.
2. **Named Entity Recognition**: Features like word shape ('New York' capitalized), previous tag, context.
3. **Sentiment analysis**: Features like presence of words ('great', 'terrible').
4. **Word sense disambiguation**: Features like surrounding words (our 'bank' example).

Limitations

1. Training can be computationally expensive.
2. Requires careful feature engineering.
3. Best results with lots of data + strong features.

Random fields

1. A random field is fundamentally a mathematical framework for representing the joint probability distribution over a set of random variables that are spatially or temporally related.
2. Think of it as a sophisticated way to model how different pieces of information influence each other.
3. In formal terms, a random field F is a collection of random variables $\{F_t : t \in T\}$ where each F_t is a random variable indexed by elements in some space T.
4. For NLP, this space T typically represents positions in a text sequence.

Types of random fields in NLP

1. **Markov Random Fields (MRFs)**: These model joint probability distributions over undirected graphs. Each variable depends on its immediate neighbors according to the Markov property.
2. **Conditional Random Fields (CRFs)**: These are discriminative models that directly model the conditional probability P(Y|X) where Y is our output sequence (like POS tags) and X is our input sequence (like words).